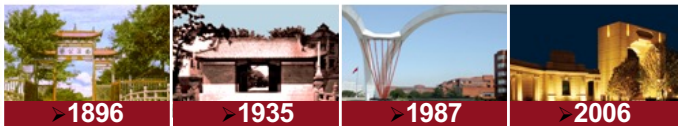
上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



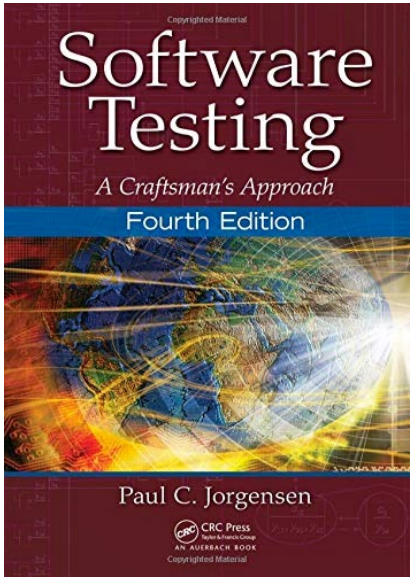
Software Testing
软件测试

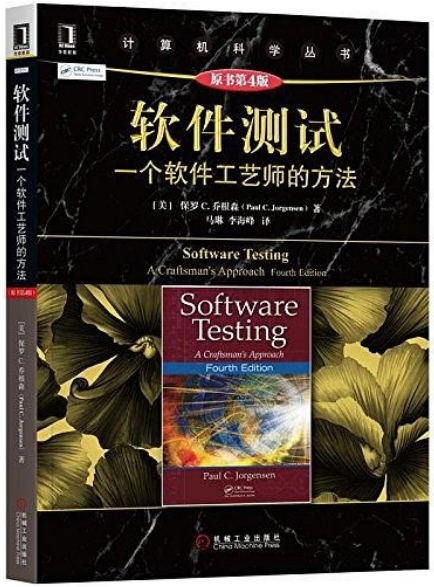
Jianguo Yao

2025-02

Course information

- Credits: 2
- 2 classes /week, 16 weeks
- Textbook:
  - Software Testing，A Craftsman's Approach，Fourth Edition，Paul C. Jorgensen，2014
  - [美] 保罗 C.乔根森 著；李海峰 译软件测试：一个软件工艺师的方法（原书第4版），机械工业出版社，2017
- Reference book:
  - 《软件测试:第3版》，人民邮电出版社，2011
  - 《软件测试：第2版》， 机械工业出版社，2002
  - "Foundation of software testing", Aditya P. Mathur,机械工业出版社, 2011年





- **The grade of the course will be determined as follows:**

  Projects 1~4：40%,

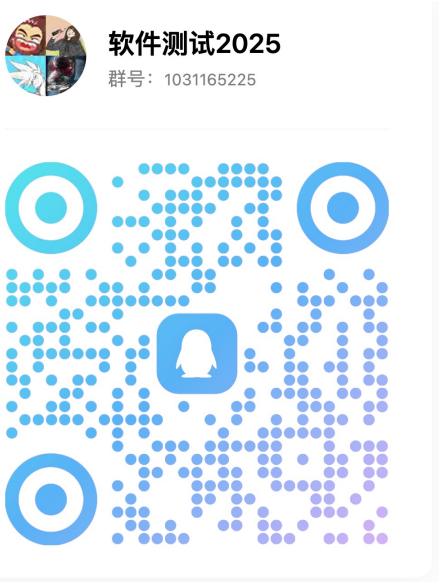
  Usual test ： 10%

  Final exam: 50%

- **The students (105) will collaborate in groups (no more than 4 students)**
- Class Prerequisites : **Java/ C++, Software Engineering**

FYI

姚建国: 软件大楼，1307 室
彭　博：软件大楼，1411室



TA：顾琪琦 qiqi.gu@sjtu.edu.cn
　　谢佳航 1351533120@sjtu.edu.cn

软件大楼 1411室
答疑形式：QQ群以及Canvas网站
答疑时间: 周三14:00~16:00



Lecture 1

Introduction of software testing

Outline

1.1 Importance of software testing

1.2 Concepts and Glossary

1.2.1 Goals of testing

1.2.2 Facts About Testing

1.2.3 Glossary

1.2.4 The basic concepts of software test

1.2.5 Fault Taxonomies

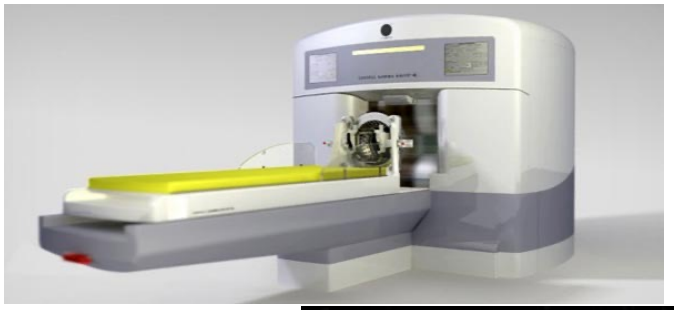


1.1 Importance of software testing

Software is Everywhere




















Mars Climate Orbiter

- Purpose: to relay signals from the Mars Polar Lander once it reached the surface of the planet
- Disaster: smashed into the planet instead of reaching a safe orbit
- Why: Software bug - failure to convert English measures to metric values
  - $165M
  - **1mile = 1609M**





Shooting Down of Airbus 300

- In 1988, US Vicennes shot down Airbus 300
- Mistook airbus 300 for a F-14
- 290 people dead
- Why: Software bug - cryptic and misleading output displayed by the tracking software





THERAC-25 Radiation Therapy

- THERAC-25, a computer controlled radiation-therapy machine
- 1986: two cancer patients at the East Texas Cancer Center in Tyler received fatal radiation overdoses
- Why: Software bug - mishandled race condition (i.e., miscoordination between concurrent tasks)



London Ambulance Service

- London Ambulance Service Computer Aided Dispatch (LASCAD)
  - Purpose: automate many of the human-intensive processes of manual dispatch systems associated with ambulance services in the UK
- Functions: Call taking
  - Failure of the London Ambulance Service on 26 and 27 November 1992



"Nice of You to Turn Up"

- Load increased
- Emergencies accumulated
- System made incorrect allocations
  - more than one ambulance being sent to the same incident
  - the closest vehicle was not chosen for the emergency
- At 23:00 on October 28 the LAS eventually instigated a backup procedure, after the death of at least 20 patients

More…

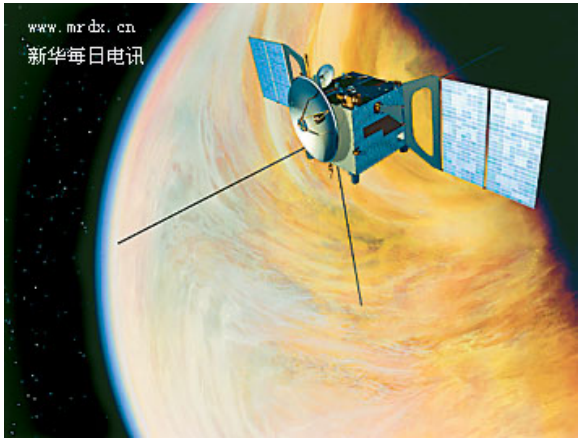
"Software and its Impact: A Quantitative Assessment," by B.W. Boehm, Datamation, 19(5), 48-59 (1973)

- Errors in medical software have caused deaths

More…

- "The development of software for ballistic-missile defense," by H. Lin, Scientific American, vol. 253, no. 6 (Dec. 1985), p. 48
  - An Air New Zealand airliner crashed into an Antarctic mountain
  - North American Aerospace Defense Command reported that the U.S. was under missile attack; traced to faulty computer software - generated incorrect signals
  - Manned space capsule Gemini V missed its landing point by 100 miles; software ignored the motion of the earth around the sun





More…

"Ghost in the Machine," Time Magazine, Jan. 29, 1990. p. 58

- **Dallas/Fort Worth air-traffic system began spitting out gibberish in the Fall of 1989 and controllers had to track planes on paper**

More…

Software Reliability: Principles & Practice, p. 25, by G. J. Myers，1978

- Apollo 8 spacecraft erased part of the computer's memory
- Eighteen errors were detected during the 10-day flight of Apollo 14
- An error in a single FORTRAN statement resulted in the loss of the first American probe to Venus(金星)

More…

- ACM SIGSOFT Software Engineering Notes, vol. 6, no. 2, 1981
  - F-18 fighter plane crashed due to a missing exception condition
- ACM SIGSOFT Software Engineering Notes, vol. 9, no. 5, 1984
  - F-14 fighter plane was lost to uncontrollable spin, traced to tactical software

More…

- Year 1999, Y2K bug, cost 500 billion

- For saving storage of computer, In the past, the software used to store the date in two Arabia numbers. “99” means “1999”, ”00” means “1900”, not “2000”. So, when 2000 is come, Software will meet a big problem!

- To solve the Y2K Bug problem and put into the software upgrade，the global investment is about 500 billion.

More …

- 2011年7月23日晚，在温州发生动车追尾脱轨坠落事故，40人死亡。
- 原因：列车控制中心设备存在严重设计缺陷、上道使用审查把关不严、雷击导致设备故障。
- 2012年1月，中国返乡旅客导致铁路网上售票系统瘫痪 原因：春运时，数以百万、千万乃至亿计的旅客需求，使意在方便旅客购票的铁路网络售票系统无法正常工作。

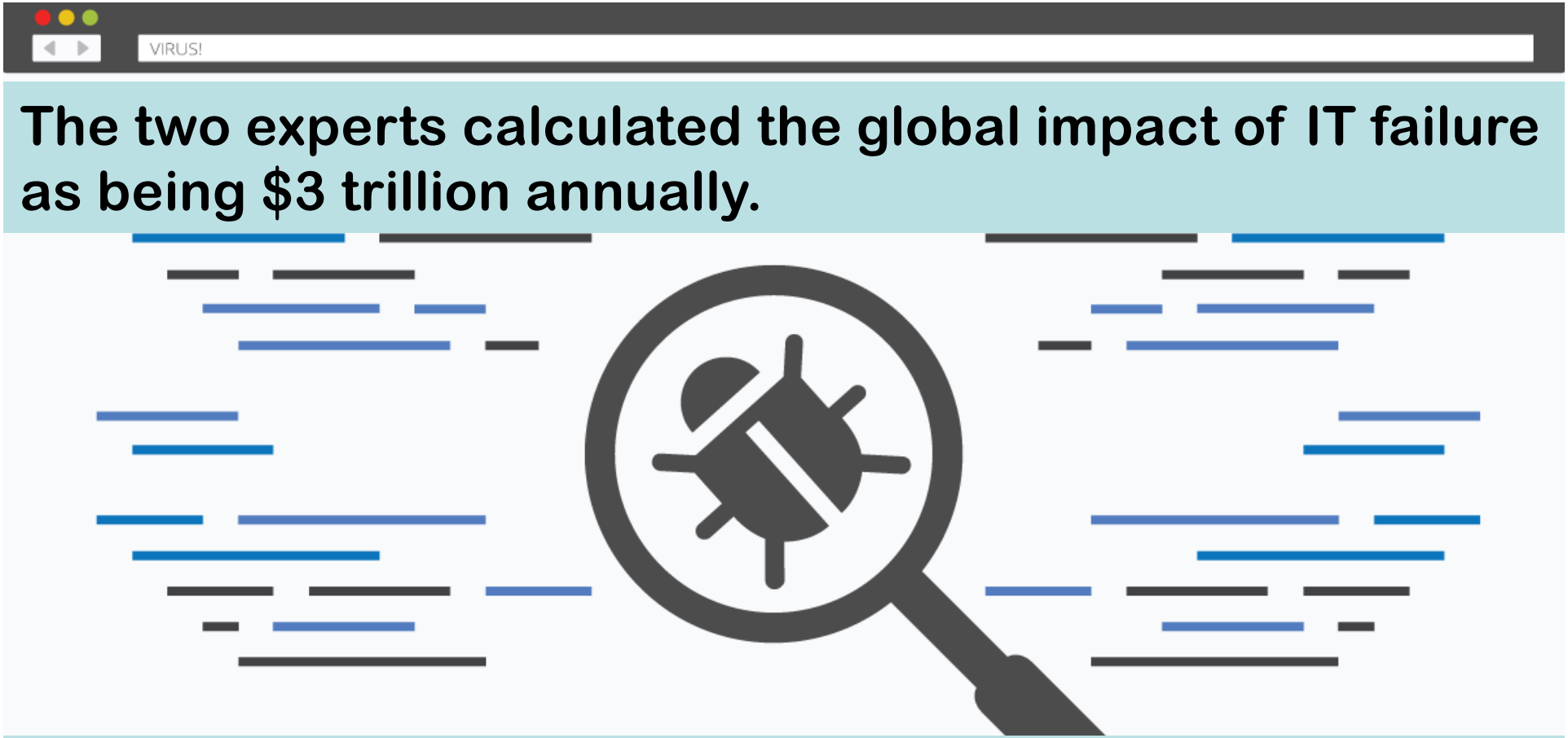
What's your experience?

- and many many more ….





Economic Impact



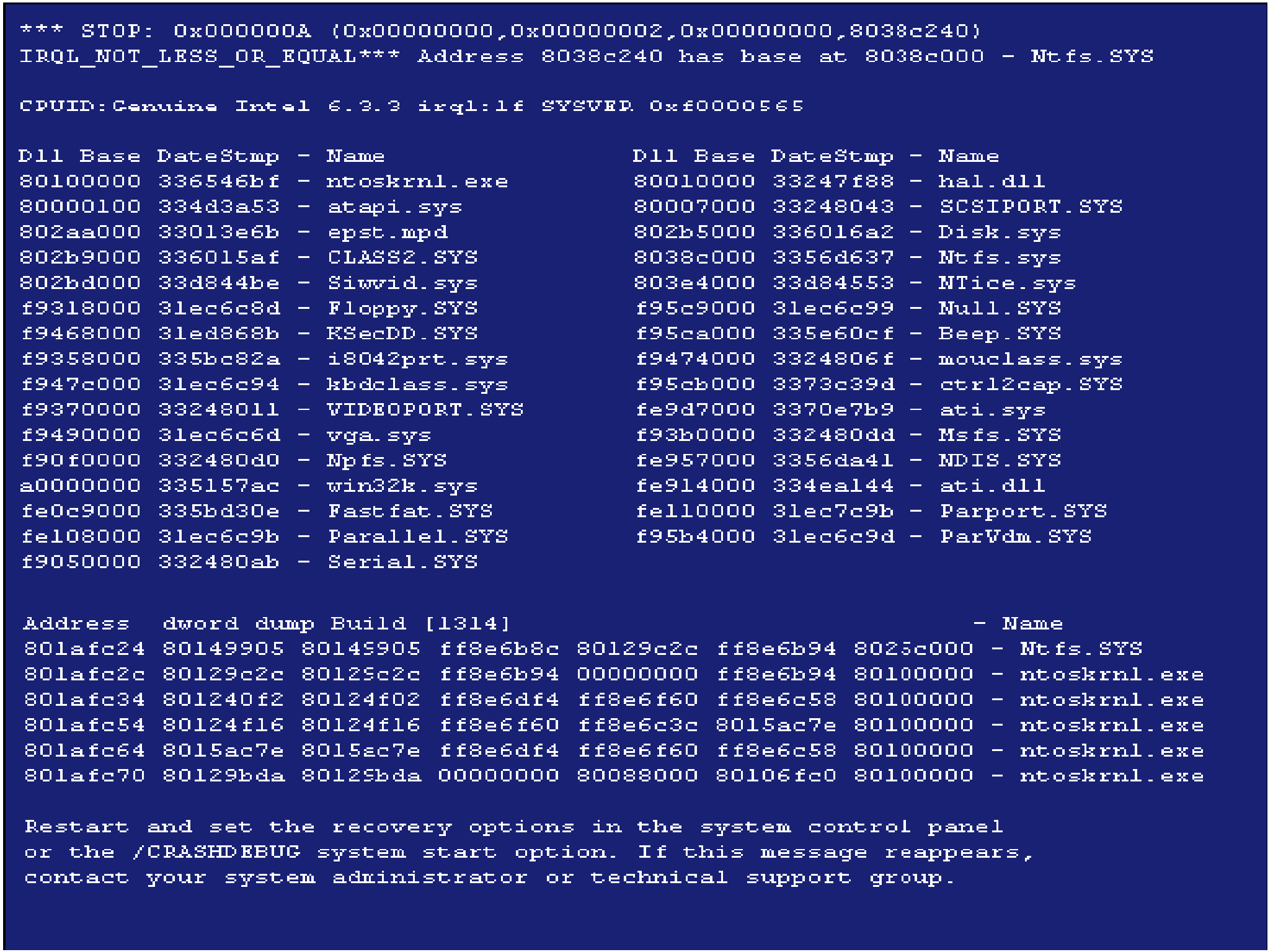
The two experts calculated the global impact of IT failure as being $3 trillion annually.

That's a staggering amount of value, 4.7 percent of global GDP, or more than the entire economic output of Germany.



Expectation

- Can't we expect software to execute correctly?
- Carefully made programs
  - 5 faults/1000 LOC
  - 1M LOC will have 5000 faults
- Windows XP has 45M LOC
  - How many faults?
  - 45 x 5000 = 225,000
- Win 8 ： 25G~30G
- Why not remove the faults?


*** STOP: 0x0000000A (0x00000000,0x00000002,0x00000000,8038c240)
IRQL_NOT_LESS_OR_EQUAL*** Address 8038c240 has base at 8038c000 - Ntfs.SYS
CPUID:Genuine Intel 6.3.3 irql:1f SYSVER 0xf0000565
Dll Base DateStmp - Name
80100000 3365546bf - ntoskrnl.exe
80000100 334d3a53 - atapi.sys
802aa000 33013e6b - epst.mpd
802b9000 336015af - CLASS2.SYS
802bd000 33d844be - Siwvid.sys
f9318000 31ec6c8d - Floppy.SYS
f9468000 31ed868b - KSecDD.SYS
f9358000 335bc82a - i8042prt.sys
f947c000 31ec6c94 - kbdclass.sys
f9370000 332480l1 - VIDEOPORT.SYS
f9490000 31ec6c6d - vga.sys
f90f0000 332480d0 - Npfs.SYS
a0000000 335157ac - win32k.sys
fe0c9000 335bd30e - Fastfat.SYS
fe108000 31ec6c9b - Parallel.SYS
f9050000 332480ab - Serial.SYS
Dll Base DateStmp - Name
80010000 33247f88 - hal.dll
80007000 33248043 - SCSIPORT.SYS
802b5000 336016a2 - Disk.sys
8038c000 3356d637 - Ntfs.sys
803e4000 33d84553 - NTice.sys
f95c9000 31ec6c99 - Null.SYS
f95ca000 335e60cf - Beep.SYS
f9474000 3324806f - mouclass.sys
f95cb000 3373c39d - ctrl2cap.SYS
fe9d7000 3370e7b9 - ati.sys
f93b0000 332480dd - Msfs.SYS
fe957000 3356da41 - NDIS.SYS
fe914000 334ea144 - ati.dll
fe110000 31ec7c9b - Parport.SYS
f95b4000 31ec6c9d - ParVdm.SYS
Address dword dump Build [1314] - Name
801afc24 80149905 80149905 ff8e6b8c 80129c2c ff8e6b94 8023c000 - Ntfs.SYS
801afc2c 80129c2c 80129c2c ff8e6b94 00000000 ff8e6b94 80100000 - ntoskrnl.exe
801afc34 801240f2 80124f02 ff8e6df4 ff8e6f60 ff8e6c58 80100000 - ntoskrnl.exe
801afc54 80124f16 80124f16 ff8e6f60 ff8e6c3c 8015ac7e 80100000 - ntoskrnl.exe
801afc64 8015ac7e 8015ac7e ff8e6df4 ff8e6f60 ff8e6c58 80100000 - ntoskrnl.exe
801afc70 80129bda 80129bda 00000000 80088000 80106fc0 80100000 - ntoskrnl.exe
Restart and set the recovery options in the system control panel
or the /CRASHDEBUG system start option. If this message reappears,
contact your system administrator or technical support group.

They Make Mistakes Too!

- Even good design cannot guarantee a perfect product
- Need testing of all products including software

How get high quality software

Two ways：

- Process management
  - CMM,PSP,TSP
  - Engineering process

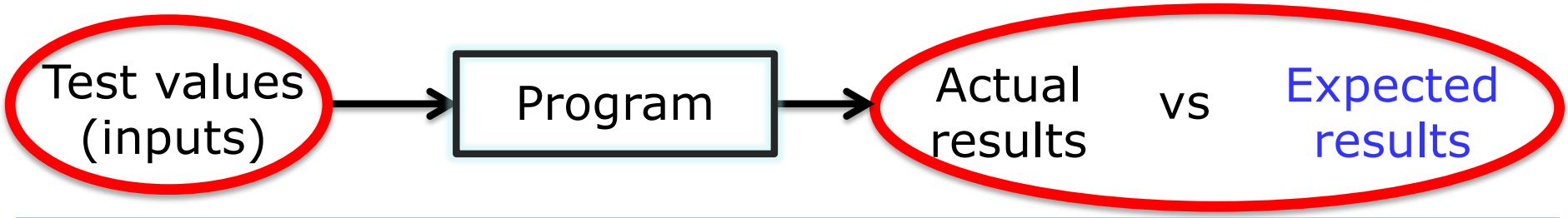
- Testing

What is Software Testing?

- **Testing = process of finding input values to check against a software (*focus of this course*)**

  Test case consists of test values and expected results

- **Debugging = process of finding a fault given a failure**



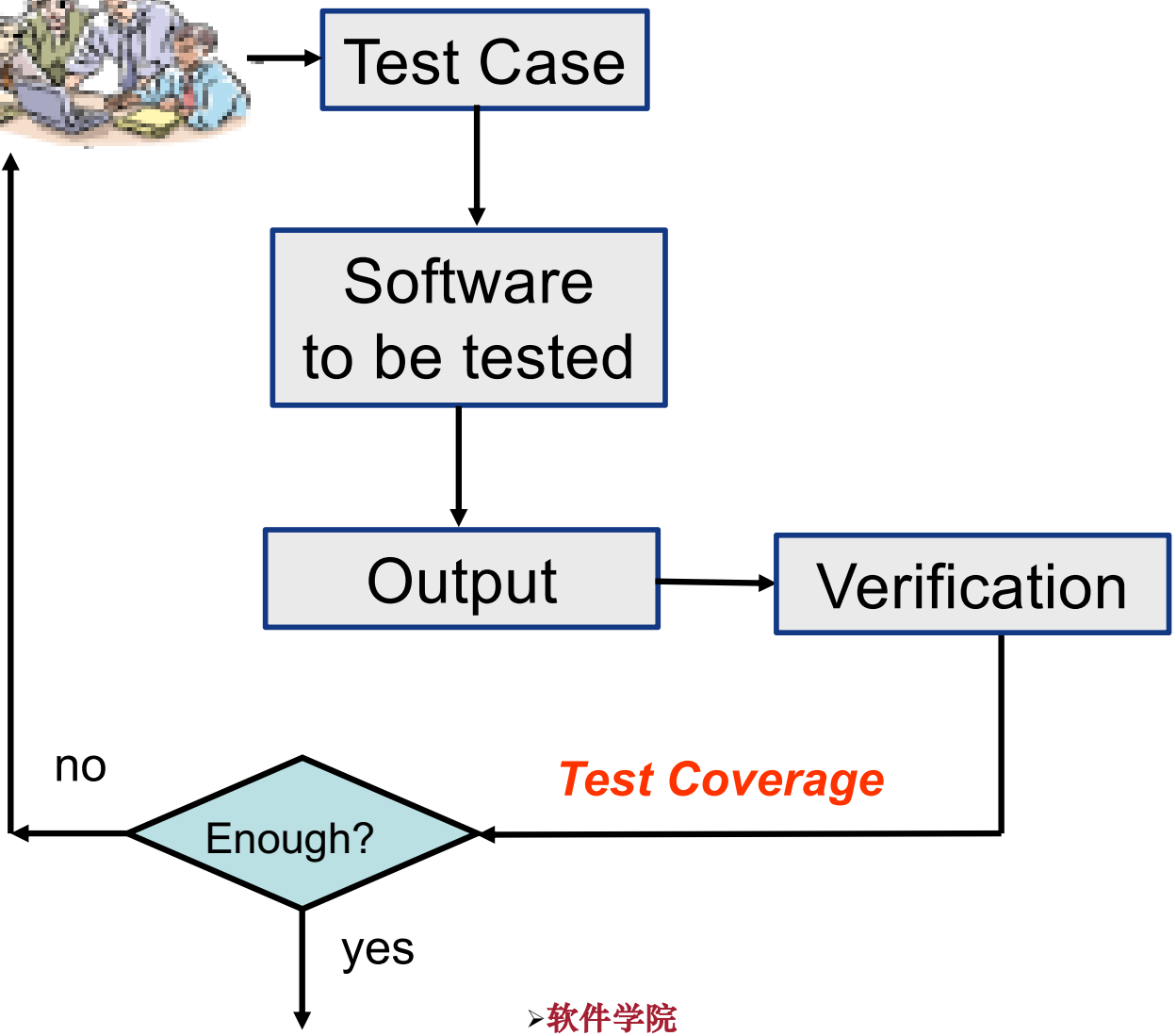
1. Testing is fundamentally about choosing finite sets of values from the input domain of the software being tested

2. Given the test inputs, compare the actual results with the expected results

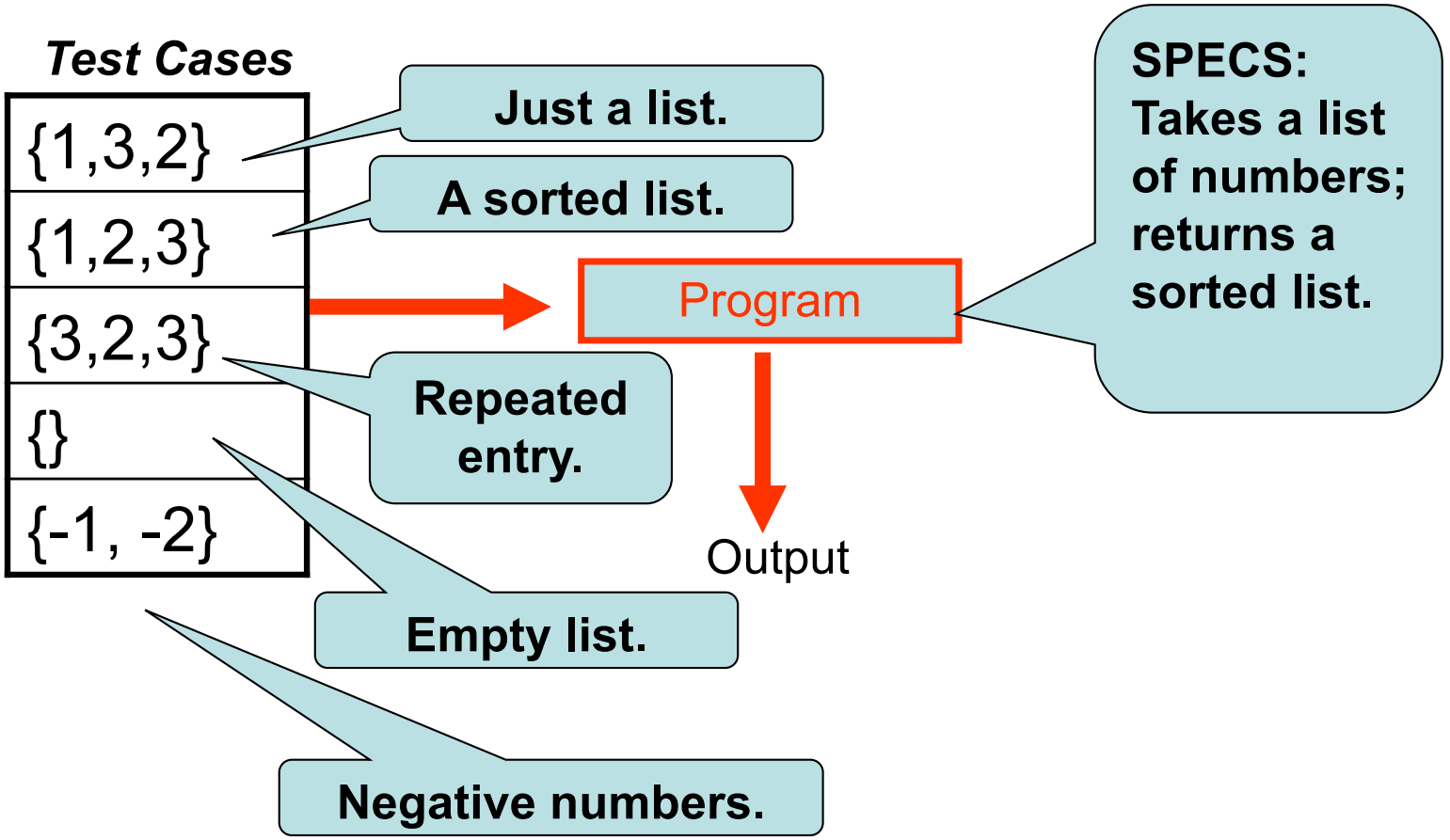
When to Stop?

Test Case Generation





A Real Testing Example





Discussion

- Different Software Types
  - Object-Oriented
  - Component-Based
  - Concurrent
  - Distributed
  - Graphical-User Interfaces
  - Web
- Different goals of testing
  - Function
  - Performance
  - Security
  - Usability
  - Correctness
  - …



1.2 Concepts and Glossary

1.2.1 Goals of testing

- Testing is a process of executing a program with the intent of finding an error.
- A good test is one that has a high probability of finding an as yet undiscovered error.
- A successful test is one that uncovers an as yet undiscovered error.
- The objective is to design tests that systematically uncover different classes of errors and do so with a minimum amount of time and effort.



- Demonstrate that software functions appear to be working according to specification.
- That performance requirements appear to have been met.
- Data collected during testing provides a good indication of software reliability and some indication of software quality.

Testing cannot show the absence of defects, it can only show that software defects are present.

1.2.2 Facts About Testing

- Question “does program P obey specification S” is undecidable!
- Every testing technique embodies some compromise between accuracy and computational cost

- Facts
  - Inaccuracy is not a limitation of the technique. It is theoretically impossible to devise a completely accurate technique
  - Every practical technique must sacrifice accuracy in some way

Why?

- **The current development methods of software are:**
  - Depend on people's intelligence
  - Specification is informal or even chaos
  - Software development tools are not self-contained
  - Software development process is elective and without standards on detail

在当前的软件开发方法学指导下开发软件，软件测试仍是保证软件质量和可靠性的重要手段。

Cost/benefit

- Testing takes more than 50% of the total cost of software development
  - More for critical software
- Software quality will become the dominant success criterion



The understanding of software test

- Exhausted test is impossible to realized.
- Software test can not ensure to uncover all errors of a software.
- Software test can not instead of other quality insurance methods

软件测试：巨大的效益和艰苦的工作并存

Some examples

1 Exception

```
if(state = = 0)
{    …
 }
if(state = = 1)
{…} ;
```

2 Data overflow

```
Int count( int &a, int month_num)
{ int  month_income = 0, i;
    for( i = 0; i< month_num; i++)
          month_income+=a[i];
          return month_income;
}
//1000<a[i]<10000
```

Some examples

3 Beyond array boundary

```
void input(…)
{
…
   strcopy( stu_name,name)
…
}
//char stu_name[20];
```

4 Data is inaccurate

```
Every data type is
 limited by it's bits.


Such as float variable
accurate is 7 bit, so
123456799.00 is equal to
123456711.00.
```

1.2.3 Glossary

- **Error (ISO)(错误)**
  - A good synonym is mistake. Errors tend to propagate;
- **Fault(故障)**
  - A fault is the result of an error. Defect (see the ISTQB Glossary) is a good synonym for fault.
- **Failure (IEEE)(失效)**
  - The inability of a system or component to perform its required functions within specified performance requirements.
- **Incident(事件)**
  - When a failure occurs, it may or may not be readily apparent to the user (or customer or tester). An incident is the symptom associated with a failure that alerts the user to the occurrence of a failure.



- **Exception (IEEE)(例外/越界)**
  - An event that causes suspension of normal program operation. Types include addressing exception, data exception, operation exception, overflow exception, protection exception, underflow exception
- **Anomaly (IEEE)(异常)**
  - Anything observed in the documentation or operation of software, that deviates from expectations based on previously verified software products or reference documents



Example

```
public static int numZero (int[] arr)
{  // Effects: If arr is null throw NullPointerException
   // else return the number of occurrences of 0 in arr
   int count = 0;
   for (int i = 1; i < arr.length; i++)
      if (arr[i] == 0)
         count++;
   return count;
}
```

There is a simple fault in **numZero**

Where is the fault location in the source code?

How would you fix it?

Can the fault location be reached? How does it corrupt program state? Does it always corrupt the program state?

If the program state is corrupted, does **numZero** fail? How?



Example – Let's Analyze

```
public static int numZero (int[] arr)
{  // Effects: If arr is null throw NullPointerException
   // else return the number of occurrences of 0 in arr
   int count = 0;
   for (int i = 1; i < arr.length; i++)
      if (arr[i] == 0)
         count++;
   return count;
}
```

Fault: a defect in source code

`i = 1` [should start searching at 0, not 1]

Error: erroneous program state caused by execution of the defect

`i` becomes 1 [array entry 0 is not ever read]

Failure: propagation of erroneous state to the program outputs

Happens as long as `arr.length > 0` and `arr[0] = 0`

Example – Test Cases

```
public static int numZero (int[] arr)
{  // Effects: If arr is null throw NullPointerException
   // else return the number of occurrences of 0 in arr
   int count = 0;
   for (int i = 1; i < arr.length; i++)
      if (arr[i] == 0)
         count++;
   return count;
}
```

Fault: `i = 1` [should start searching at 0, not 1]

Test 1: [4, 6, 0], expected 1

Error: `i` is 1, not 0, on the first iteration

Failure: none

Test 2: [0, 4, 6], expected 1

Error: `i` is 1, not 0, error propagates to the variable `count`

Failure: `count` is 0 at the return statement



Example – State Representation

```
   public static int numZero (int[] arr)
   {  // Effects: If arr is null throw NullPointerException
      // else return the number of occurrences of 0 in arr
L1    int count = 0;
L2    for (int i = 1; i < arr.length; i++)
L3       if (arr[i] == 0)
L4          count++;
L5    return count;
   }
```

Assume that we want to represent program states using the notation **<var_1 = v_1, …, var_n = v_n, PC = program counter>**

Sequence of states in the execution of **numZero({0, 4, 6})**

1: < arr={0, 4, 6}, PC=[**int count=0** (L1)] >
2: < arr={0, 4, 6}, **count=0**, PC=[**i=1** (L2)] >
3: < arr={0, 4, 6}, **count=0**, **i=1**, PC=[**i<arr.length** (L2)] >

…

< arr={0, 4, 6}, **count=0**, PC=[**return count;** (L5)] >

Example – Error State

Error state

The first different state in execution in comparison to an execution to the state sequence of what would be the correct program

If the code had i=0 (correct program), the execution of **`numZero({0, 4, 6})`** would be

1: < arr={0, 4, 6}, PC=[**`int count=0`** (L1)] >
2: < arr={0, 4, 6}, **`count=0`**, PC=[**`i=0`** (L2)] >
3: < arr={0, 4, 6}, **`count=0`**, **`i=0`**, PC=[**`i<arr.length`** (L2)] >
…

Instead, we have

1: < arr={0, 4, 6}, PC=[**`int count=0`** (L1)] >
2: < arr={0, 4, 6}, **`count=0`**, PC=[**`i=1`** (L2)] >
3: < arr={0, 4, 6}, **`count=0`**, **`i=1`**, PC=[**`i<arr.length`** (L2)] >
…

The first error state is immediately after **`i=1`** in line L2

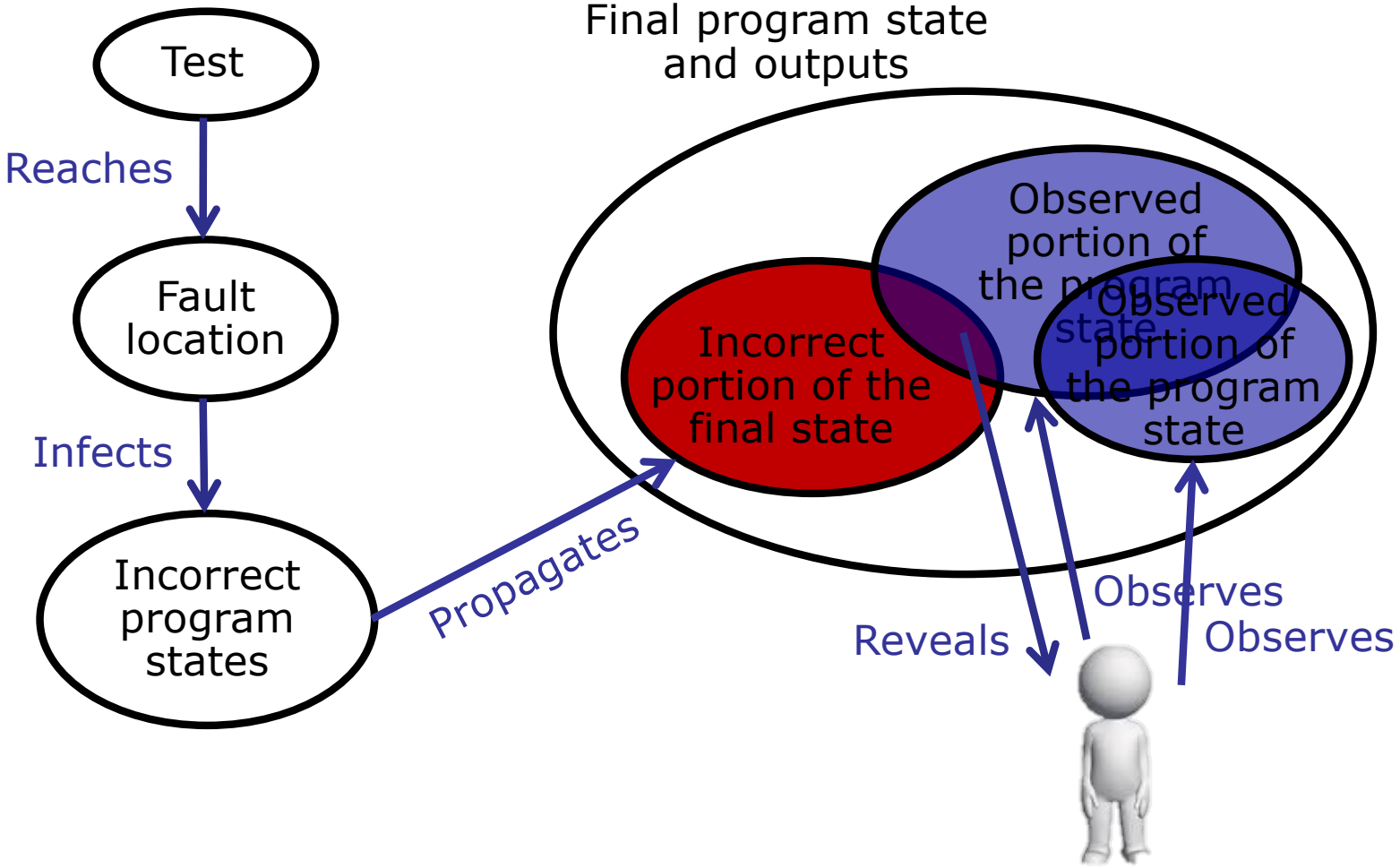


RIPR Model

Four conditions necessary for a failure to be observed

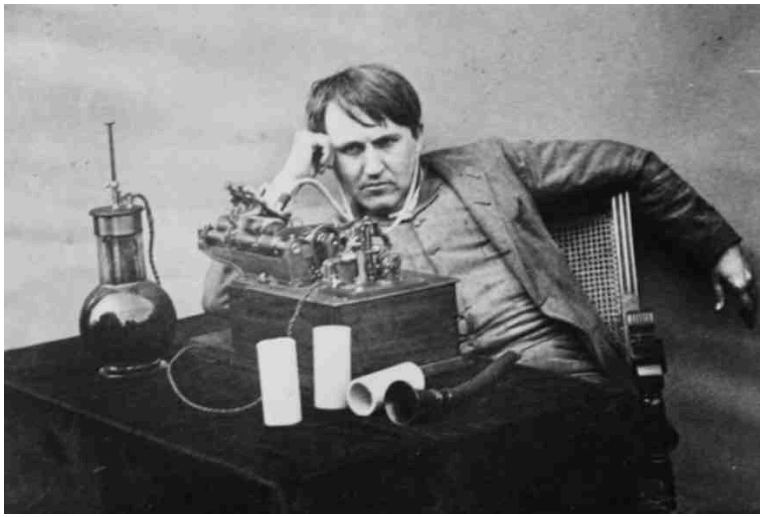
- **R**eachability
  - The fault is reached
- **I**nfection
  - Execution of the fault leads to an incorrect program state (error)
- **P**ropagation
  - The infected state must cause the program output or final state to be incorrect (failure)
- **R**evealability
  - The tester must observe part of the incorrect portion of the program state

RIPR Model


Test
Reaches
Fault location
Infects
Incorrect program states
Propagates
Final program state and outputs
Incorrect portion of the final state
Observed portion of the program state
Observed portion of the program state
Reveals
Observes
Observes

About “bug”

“ ‘Bug’ – *as such little faults and difficulties are called – show themselves, and months of anxious watching, study, and labor are requisite before commercial success – or failure – is certainly reached.” [Thomas Edison, 1878]*



[Did You Know? Edison Coined the Term “Bug”, http://theinstitute.ieee.org/tech-history/technology-history/did-you-know-edison-coined-the-term-bug, IEEE 2013]

Bug is used informally.



Fault? Error? Or failure?

This course will try to use words that have precise, defined, and unambiguous meaning



Example – RIPR (Error, No Failure)

```
   public static int numZero (int[] arr)
   {  // Effects: If arr is null throw NullPointerException
      // else return the number of occurrences of 0 in arr
L1    int count = 0;
L2    for (int i = 1; i < arr.length; i++)
L3       if (arr[i] == 0)
L4          count++;
L5    return count;
   }
```

Revisit the example, apply RIPR to design tests that

- Reach a fault (i.e., execute the fault)
- Cause the program state to be incorrect (i.e., error)
- Does not propagate (i.e., no failure)
  - One possible test is [4, 6, 0] – now, design some more
- How does RIPR model help designing tests?



Example – RIPR (Error, Failure)

```
   public static int numZero (int[] arr)
   {  // Effects: If arr is null throw NullPointerException
      // else return the number of occurrences of 0 in arr
L1    int count = 0;
L2    for (int i = 1; i < arr.length; i++)
L3       if (arr[i] == 0)
L4          count++;
L5    return count;
   }
```

Revisit the example, apply RIPR to design tests that

- Reach a fault (i.e., execute the fault)
- Cause the program state to be incorrect (i.e., error)
- Propagate (i.e., failure)
  - One possible test is [0, 4, 6] – now, design some more
- How does RIPR model help designing tests?



1.2.4 The basic concepts of software test

- Basic principle of software test (Myers principle）
- Independent principle
- Completive principle
- The types of software test

Myers principle

- Early test and continual test
- Avoid test software by programmer him/her self
- Test case = input data + expected result
- Test case should include expected（valid）input data and unexpected（invalid）input data
- Test program = what it should to do + what it should not do
- The probability of existing errors is direct proportion to the errors have being uncovered.
- Check each test result roundly
- Software test is an most innovational work

Independent principle

- Software test should be execute by the group which is independent from development group.

第三方测试

Completive principle

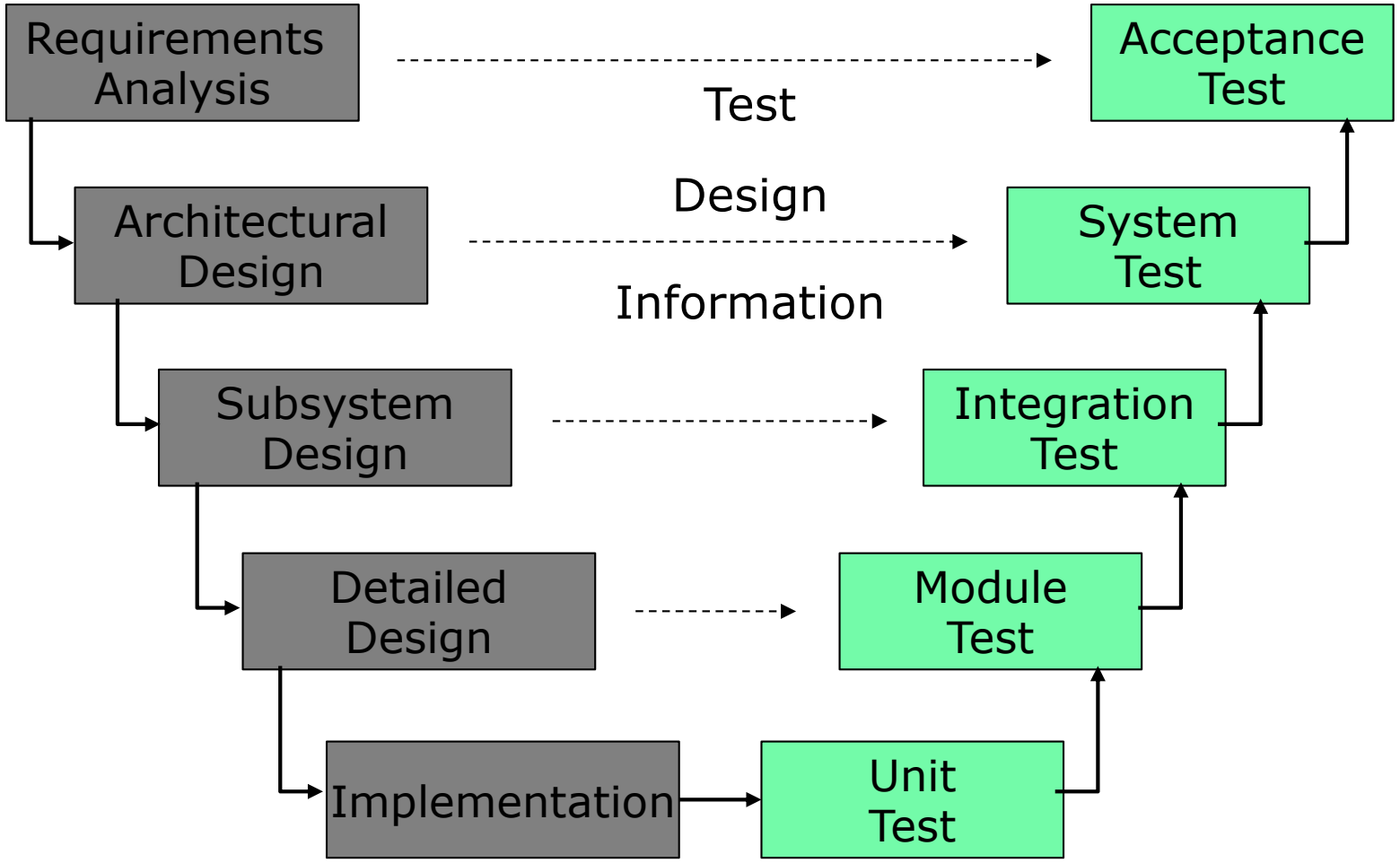
- Complete all test cases according to the test plan
- Use stated test case generate methods.
- Reach expected object (such as coverage criteria）
- Draw a curve for analyze, which shows the errors for different time period.
- Should uncover an expected number of errors.

Steps of software test

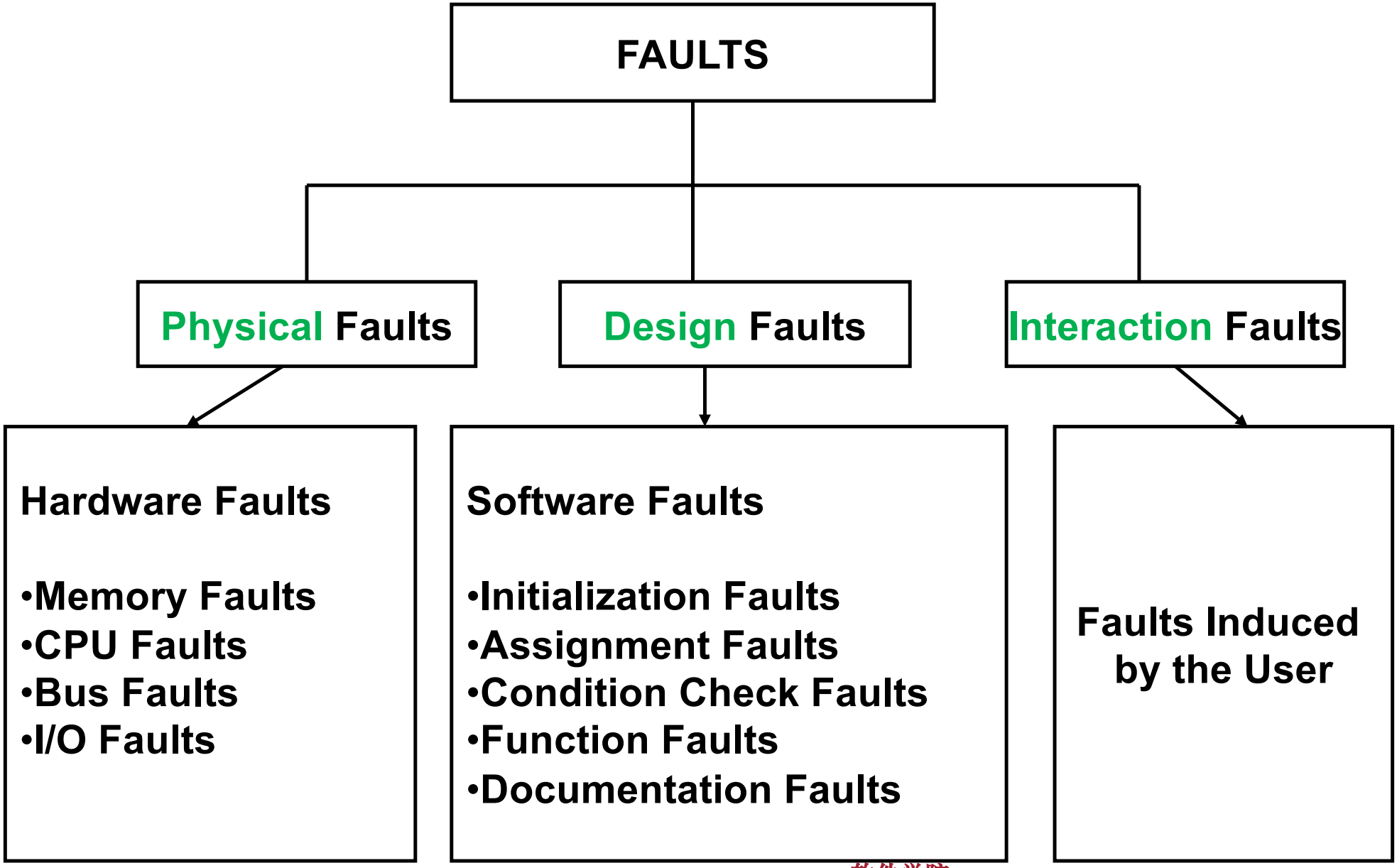


- Design test plan
- Generate test case
- Establish test environment
- Execute test case
- Evaluate test result
- Complete test documentation : test plan, test explain and test report

Testing and SW Development Lifecycle



1.2.5 Fault Taxonomies





Questions?